Bank–NBFI Systemic Risk Analysis

This notebook walks through the full analysis pipeline:

1. **Data**: Generate synthetic data (or load real data)
2. **Network analysis**: Map bank-NBFI interlinkages
3. **Systemic risk**: CoVaR, MES, SRISK, Diebold-Yilmaz connectedness
4. **DCC-GARCH**: Time-varying correlations between sectors
5. **VaR amplification**: Procyclicality tests and fire-sale simulation
6. **NBFI subsector models**: LDI margin spiral, MMF run, HF deleveraging
7. **Global financial cycle**: Factor extraction and amplification regressions

In [1]:
```
import warnings
warnings.filterwarnings('ignore')

import sys, os
sys.path.insert(0, os.path.abspath('..'))

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline
plt.rcParams.update({'figure.figsize': (12, 6), 'figure.dpi': 100})
```

1. Data

In [2]:
```
from src.data.build_dataset import generate_synthetic_data

data = generate_synthetic_data(seed=42)
institutions = data['institutions']
returns = data['returns']
exposures = data['exposures']
macro = data['macro']

print(f"Institutions: {len(institutions)} ({(institutions.sector == 'bank
print(f"Returns: {returns.shape[0]} days x {returns.shape[1]} entities")
print(f"Exposures: {len(exposures)} bilateral obs across {exposures.date.
institutions.groupby('sector').size()
```

```
Institutions: 80 (30 banks, 50 NBFIs)
Returns: 6522 days x 80 entities
Exposures: 20658 bilateral obs across 80 quarters
```

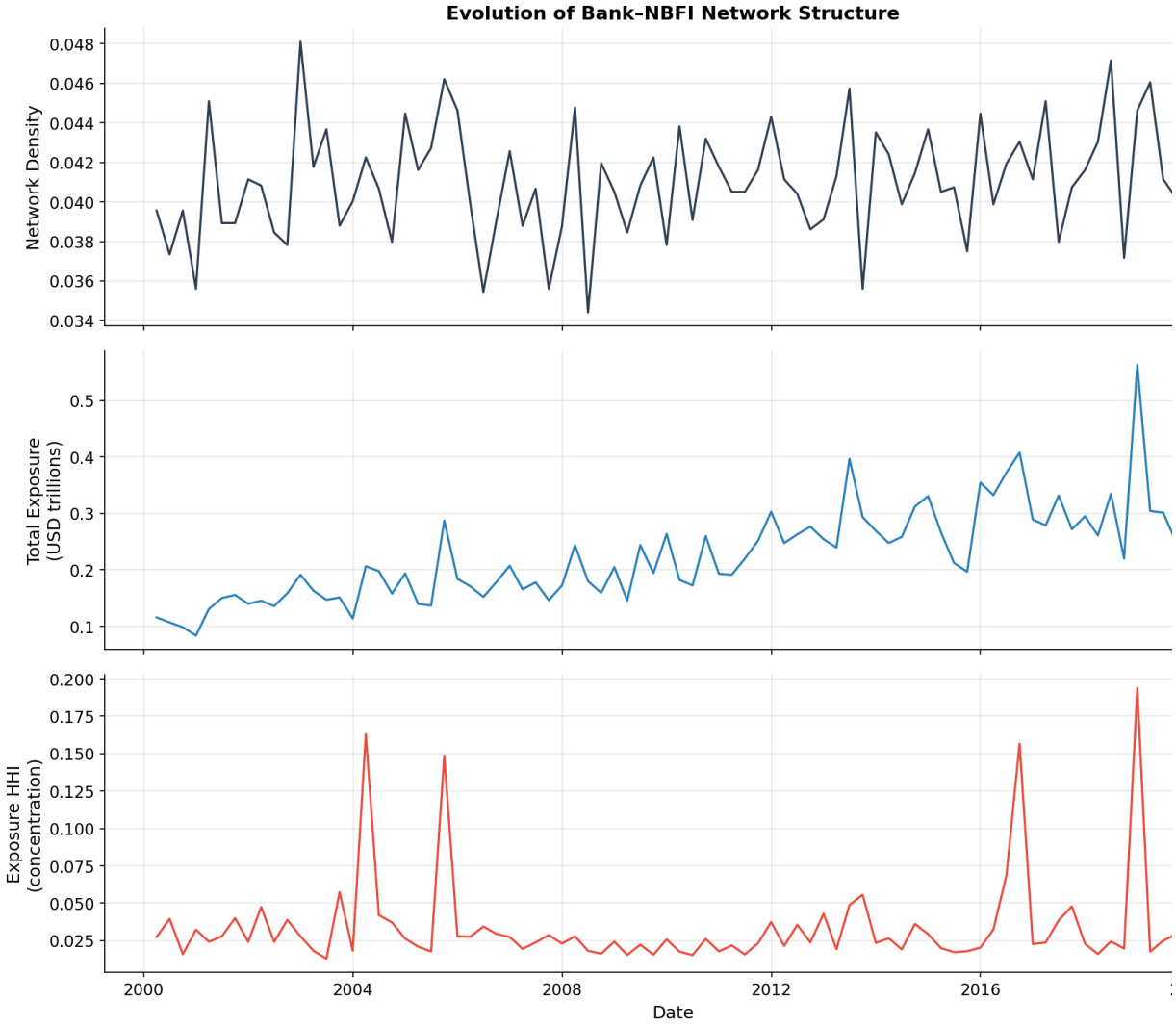
Out[2]:
```
sector
bank                30
broker_dealer        4
direct_credit        6
hedge_fund          10
insurance           11
investment_fund      6
mmf                  6
pension_fund         7
dtype: int64
```

2 . Network Analysis

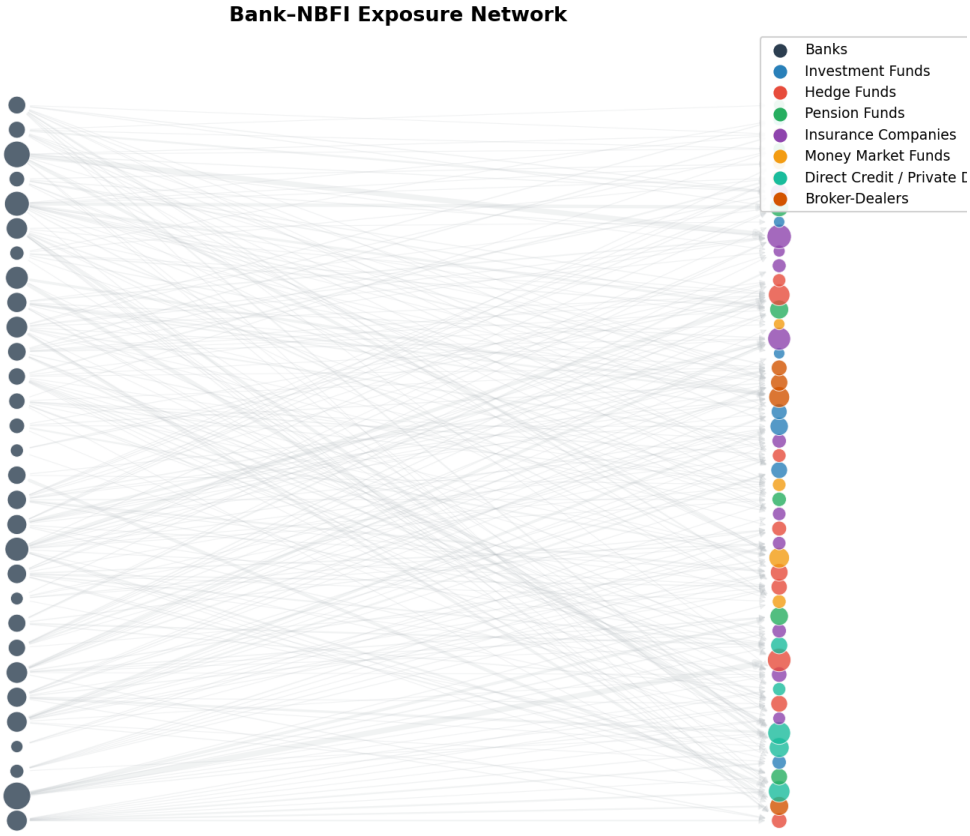
In [3]:
```
from src.analysis.network import (
    build_exposure_network, compute_centrality_measures,
    rolling_network_statistics, compute_network_statistics,
)
from src.visualization.plots import plot_exposure_network, plot_network_s

# Latest quarter network
latest_date = exposures['date'].max()
G = build_exposure_network(exposures, date=latest_date)
stats = compute_network_statistics(G)
print(f"Network: {stats['n_nodes']} nodes, {stats['n_edges']} edges, dens

# Centrality
centrality = compute_centrality_measures(G)
centrality_with_sector = centrality.merge(
    institutions[['ticker', 'sector']], left_index=True, right_on='ticker
)
print("\nTop 10 by PageRank:")
print(centrality.nlargest(10, 'pagerank')[['pagerank', 'total_strength']]
```

```
Network: 80 nodes, 267 edges, density=0.042

Top 10 by PageRank:
          pagerank  total_strength
node
NBFI_035  0.022950        13932.20
NBFI_039  0.020824        19220.30
NBFI_033  0.020582        13939.42
NBFI_024  0.020510        16374.03
NBFI_046  0.019630         9574.86
NBFI_038  0.019268         3951.09
NBFI_026  0.019201        16154.65
NBFI_016  0.019082        12812.99
NBFI_032  0.018922        12227.97
NBFI_040  0.017571        11098.99
```

In [4]:
```
# Network evolution
net_stats = rolling_network_statistics(exposures)
fig = plot_network_statistics_over_time(net_stats, save_name=None)
plt.show()
```



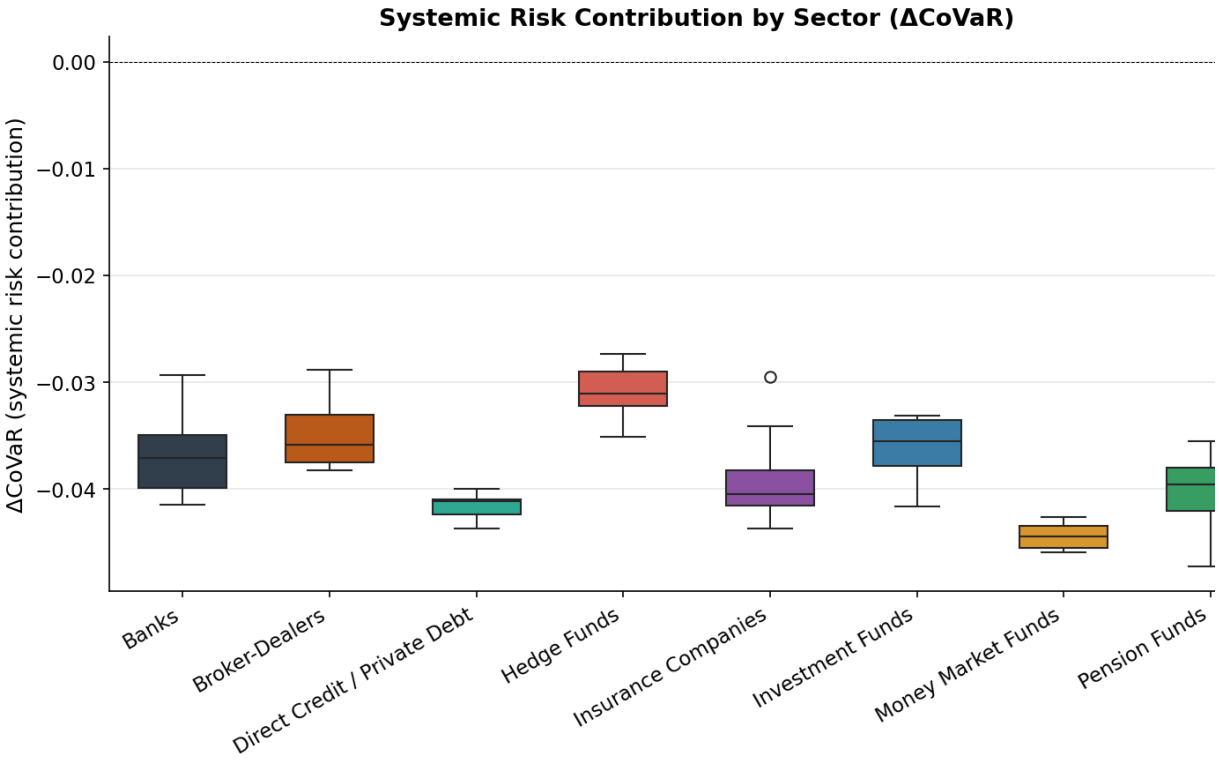
In [5]:
```
fig = plot_exposure_network(G, institutions, save_name=None)
plt.show()
```



Banks

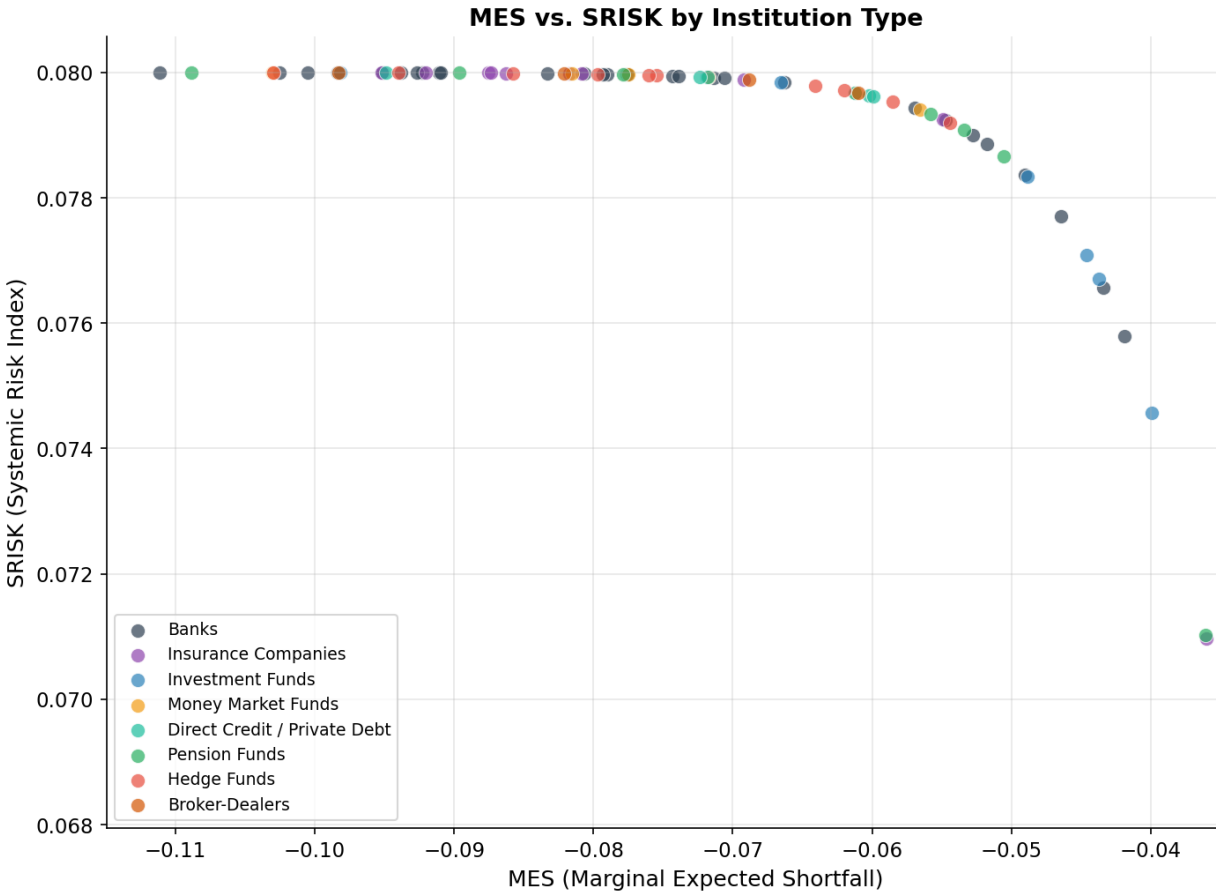
3 . Systemic Risk Measures

In [6]:
```
from src.analysis.systemic_risk import (
    compute_covar_panel, compute_mes, compute_srisk,
    compute_connectedness, aggregate_connectedness_by_sector,
)
from src.visualization.plots import (
    plot_covar_by_sector, plot_mes_srisk_scatter, plot_connectedness_heat
)

weekly = returns.resample('W').apply(lambda x: (1 + x).prod() - 1)

# CoVaR
covar_df = compute_covar_panel(weekly, institutions)
fig = plot_covar_by_sector(covar_df, save_name=None)
plt.show()
print("\nMean |ΔCoVaR| by sector:")
print(covar_df.groupby('sector')['delta_covar'].apply(lambda x: x.abs().m
```



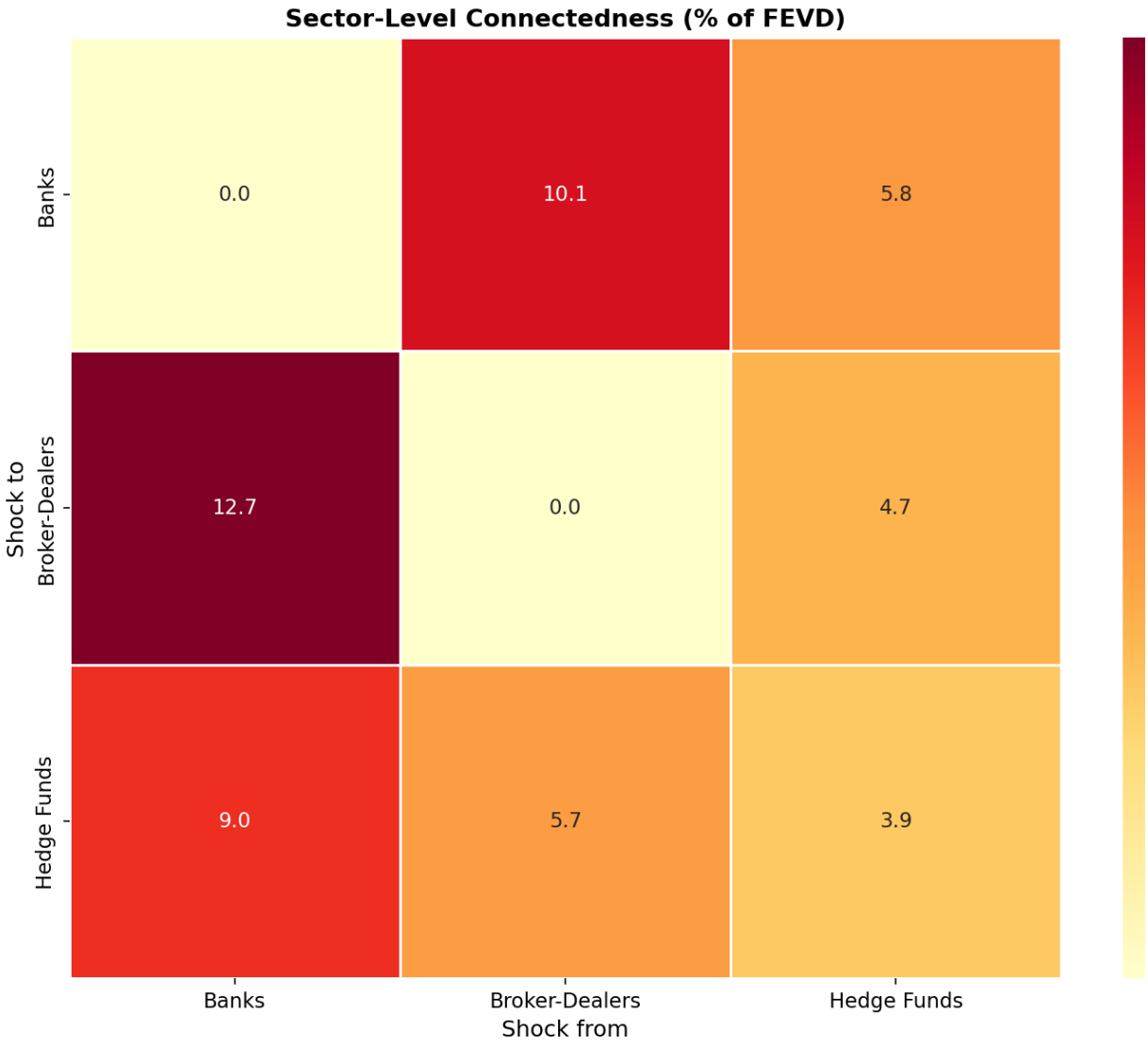

```
Mean |ΔCoVaR| by sector:
sector
hedge_fund          0.030915
broker_dealer       0.034721
investment_fund     0.036235
bank                0.036594
insurance           0.039092
pension_fund        0.040369
direct_credit       0.041625
mmf                 0.044419
Name: delta_covar, dtype: float64
```

In [7]:
```
# SRISK (combines MES)
srisk_df = compute_srisk(weekly, institutions)
fig = plot_mes_srisk_scatter(srisk_df, save_name=None)
plt.show()
```



In [8]:
```
# Diebold-Yilmaz connectedness
top_n = 12
vol_rank = weekly.std().nlargest(top_n).index.tolist()
conn = compute_connectedness(weekly[vol_rank].dropna(), lags=4, h=10)
print(f"Total connectedness: {conn['total_connectedness']:.1f}%")

sector_theta = aggregate_connectedness_by_sector(conn['theta'], instituti
fig = plot_connectedness_heatmap(sector_theta, save_name=None)
plt.show()
```

```
Total connectedness: 52.1%
```



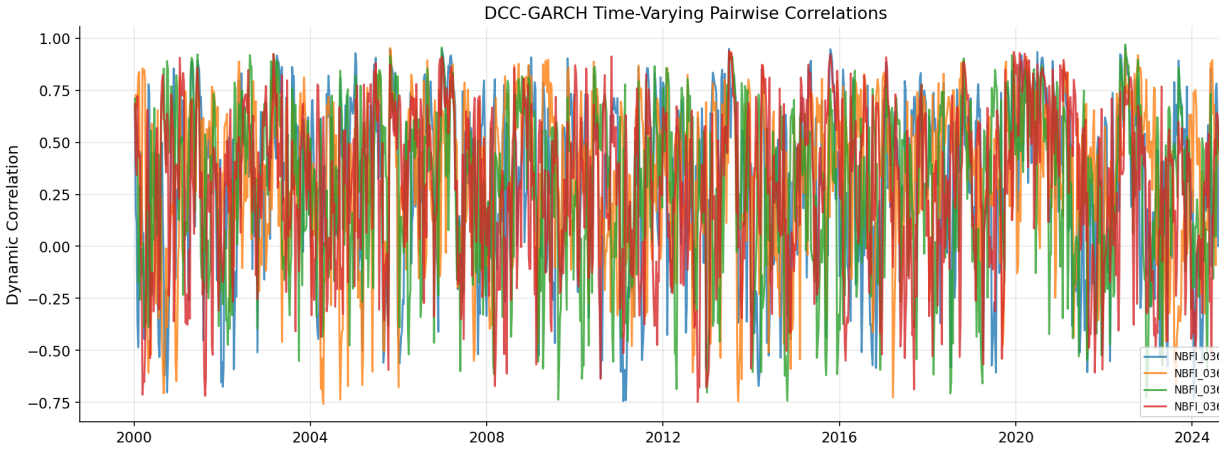
4 . DCC-GARCH: Dynamic Correlations

In [9]:
```
from src.models.dcc_garch import fit_all_garch, estimate_dcc, sector_aver

# Use a small subset for speed
subset_tickers = weekly.std().nlargest(6).index.tolist()
subset = weekly[subset_tickers].dropna()

# Step 1: Univariate GARCH
garch_result = fit_all_garch(subset)
print(f"GARCH fitted for {len(garch_result['models'])} series")

# Step 2: DCC
dcc_result = estimate_dcc(garch_result['std_resids'])
print(f"DCC parameters: a={dcc_result['a']:.4f}, b={dcc_result['b']:.4f}"
print(f"Persistence (a+b): {dcc_result['persistence']:.4f}")
```

```
GARCH fitted for 6 series
DCC parameters: a=0.3888, b=0.5458
Persistence (a+b): 0.9347
```

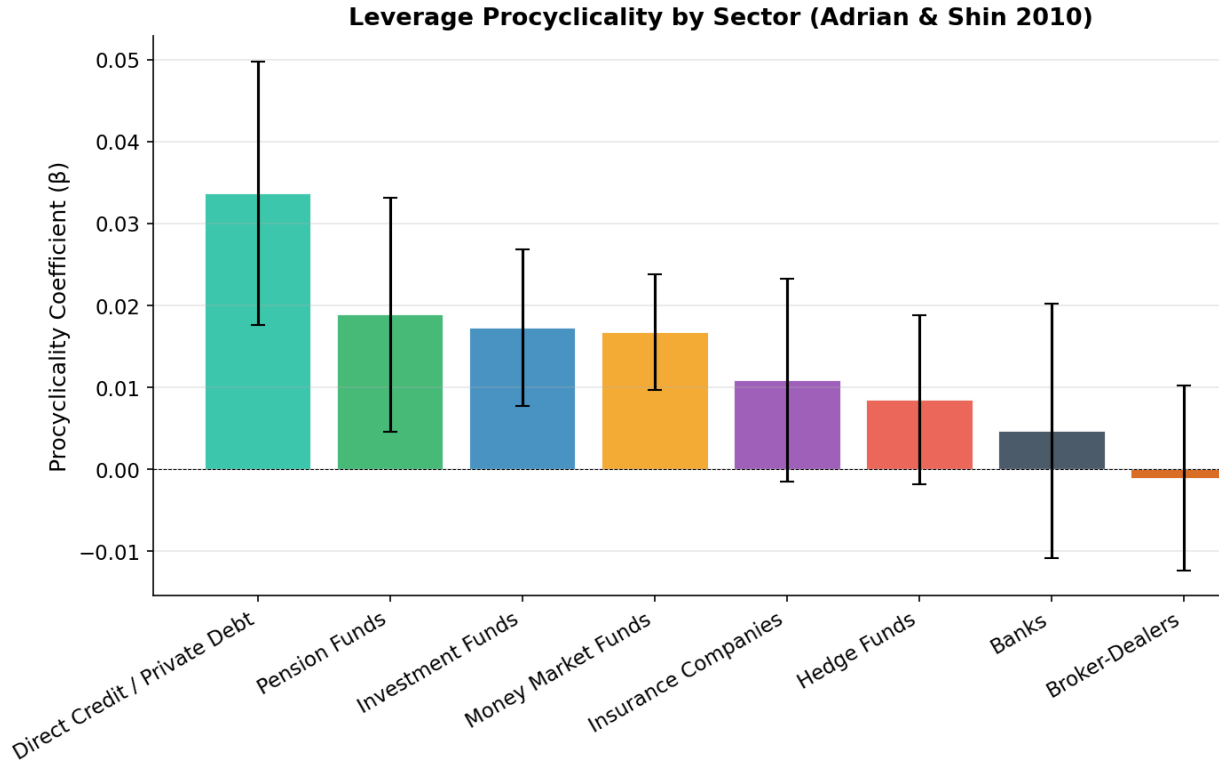
In [10]:
```
# Plot a few dynamic correlations
fig, ax = plt.subplots(figsize=(14, 5))
for i, (pair, corr) in enumerate(list(dcc_result['dynamic_correlations'].
    ax.plot(corr.index, corr.values, label=pair, alpha=0.8)
```

```
ax.set_ylabel('Dynamic Correlation')
ax.set_title('DCC-GARCH Time-Varying Pairwise Correlations')
ax.legend(fontsize=8)
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



5 . VaR Amplification

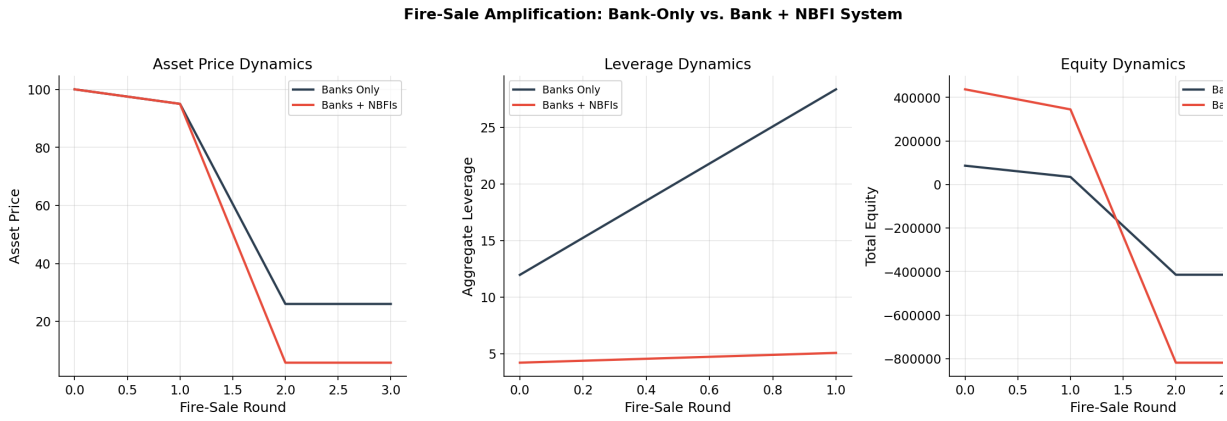
In [11]:
```
from src.models.var_amplification import (
    sector_procyclicality, run_counterfactual, amplification_ratio,
)
from src.visualization.plots import plot_fire_sale_comparison, plot_procy

# Procyclicality by sector
procyc = sector_procyclicality(returns, institutions)
fig = plot_procyclicality_by_sector(procyc, save_name=None)
plt.show()
print("\nMean procyclicality by sector:")
print(procyc.groupby('sector')['beta_procyclicality'].mean().sort_values(
```



```
Mean procyclicality by sector:
sector
direct_credit      0.033713
pension_fund       0.018871
investment_fund    0.017302
mmf                0.016742
insurance          0.010905
hedge_fund         0.008505
bank               0.004686
broker_dealer     -0.001041
Name: beta_procyclicality, dtype: float64
```

In [12]:
```python
# Fire-sale counterfactual: bank-only vs bank+NBFI
histories = run_counterfactual(initial_shock=-0.05, market_depth=5e5)
amp = amplification_ratio(histories)

print(f"Bank-only price drop:  {amp['price_drop_bank_only']:.4f}")
print(f"Bank+NBFI price drop:  {amp['price_drop_bank_nbfi']:.4f}")
print(f"Amplification ratio:   {amp['amplification_ratio']:.2f}x")
print(f"Equity loss (bank-only): {amp['equity_loss_bank_only']:.2%}")
print(f"Equity loss (bank+NBFI): {amp['equity_loss_bank_nbfi']:.2%}")

fig = plot_fire_sale_comparison(histories, save_name=None)
plt.show()
```

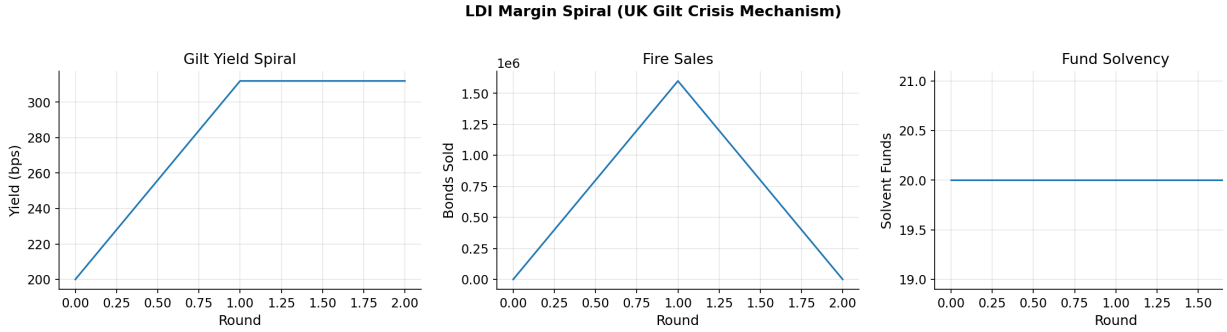
```
Bank-only price drop:  -0.7396
Bank+NBFI price drop:  -0.9419
Amplification ratio:   1.27x
Equity loss (bank-only): -580.33%
Equity loss (bank+NBFI): -287.30%
```



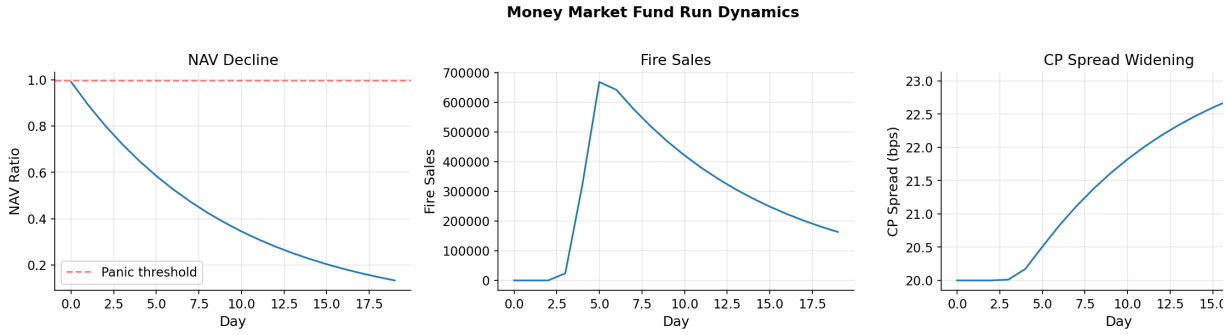
6 . NBFI Subsector Models

In [13]:
```python
from src.models.nbfi_subsectors import (
    build_pension_sector, LDIMarginSpiral,
    build_mmf_sector, MMFRunSimulation,
    build_hedge_fund_sector, PrimeBrokerageContagion,
)

# --- LDI Margin Spiral ---
pension_funds = build_pension_sector(n_funds=20)
ldi_sim = LDIMarginSpiral(pension_funds=pension_funds, market_depth=5e6)
ldi_hist = ldi_sim.run(yield_shock_bps=80)

fig, axes = plt.subplots(1, 3, figsize=(16, 4))
axes[0].plot(ldi_hist['round'], ldi_hist['yield_bps'])
```

```
axes[0].set_ylabel('Yield (bps)'); axes[0].set_title('Gilt Yield Spiral')
axes[1].plot(ldi_hist['round'], ldi_hist['total_bonds_sold'])
axes[1].set_ylabel('Bonds Sold'); axes[1].set_title('Fire Sales')
axes[2].plot(ldi_hist['round'], ldi_hist['funds_solvent'])
axes[2].set_ylabel('Solvent Funds'); axes[2].set_title('Fund Solvency')
for ax in axes: ax.set_xlabel('Round'); ax.grid(alpha=0.3)
fig.suptitle('LDI Margin Spiral (UK Gilt Crisis Mechanism)', fontweight='
plt.tight_layout(); plt.show()
```

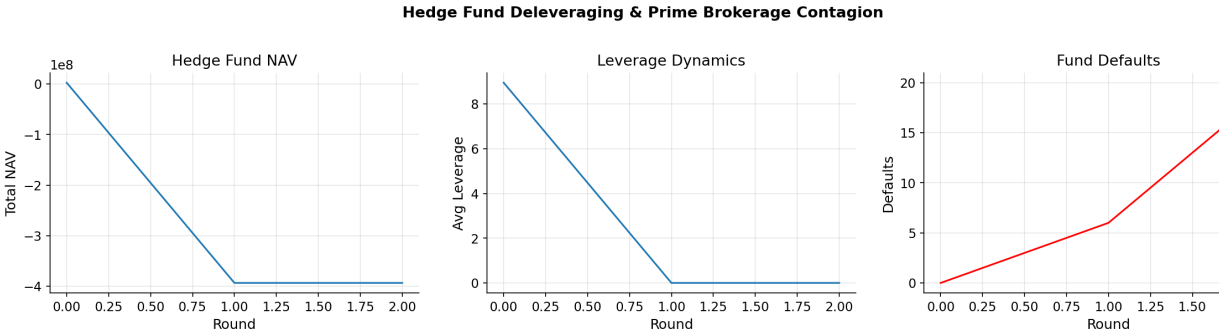


In [14]:
```
# --- MMF Run ---
mmf_funds = build_mmf_sector(n_funds=15)
mmf_sim = MMFRunSimulation(funds=mmf_funds)
mmf_hist = mmf_sim.run(initial_credit_loss_pct=0.005)

fig, axes = plt.subplots(1, 3, figsize=(16, 4))
axes[0].plot(mmf_hist['day'], mmf_hist['avg_nav_ratio'])
axes[0].axhline(0.997, ls='--', c='red', alpha=0.5, label='Panic threshol
axes[0].set_ylabel('NAV Ratio'); axes[0].set_title('NAV Decline'); axes[0
axes[1].plot(mmf_hist['day'], mmf_hist['total_fire_sales'])
axes[1].set_ylabel('Fire Sales'); axes[1].set_title('Fire Sales')
axes[2].plot(mmf_hist['day'], mmf_hist['cp_spread_bps'])
axes[2].set_ylabel('CP Spread (bps)'); axes[2].set_title('CP Spread Widen
for ax in axes: ax.set_xlabel('Day'); ax.grid(alpha=0.3)
fig.suptitle('Money Market Fund Run Dynamics', fontweight='bold', y=1.02)
plt.tight_layout(); plt.show()
```



In [15]:
```
# --- Hedge Fund Deleveraging ---
hf_sector = build_hedge_fund_sector(n_funds=20)
pb_sim = PrimeBrokerageContagion(hedge_funds=hf_sector, market_depth=3e5)
pb_hist = pb_sim.run(initial_shock=-0.04)

fig, axes = plt.subplots(1, 3, figsize=(16, 4))
axes[0].plot(pb_hist['round'], pb_hist['total_nav'])
axes[0].set_ylabel('Total NAV'); axes[0].set_title('Hedge Fund NAV')
axes[1].plot(pb_hist['round'], pb_hist['avg_leverage'])
axes[1].set_ylabel('Avg Leverage'); axes[1].set_title('Leverage Dynamics'
axes[2].plot(pb_hist['round'], pb_hist['funds_defaulted'], 'r-')
axes[2].set_ylabel('Defaults'); axes[2].set_title('Fund Defaults')
```

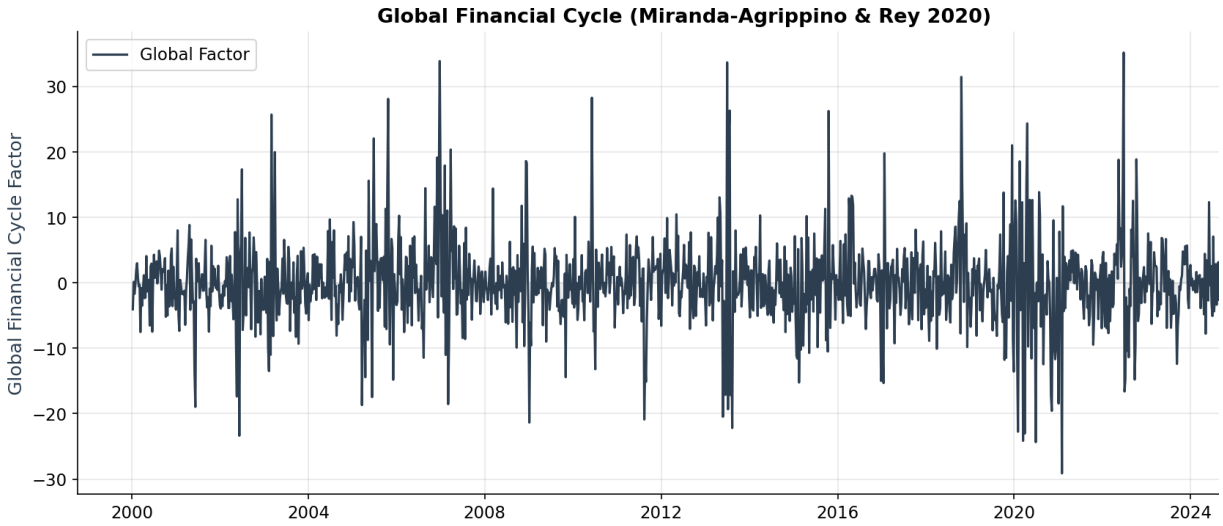
```
for ax in axes: ax.set_xlabel('Round'); ax.grid(alpha=0.3)
fig.suptitle('Hedge Fund Deleveraging & Prime Brokerage Contagion', fontw
plt.tight_layout(); plt.show()
```



7 . Global Financial Cycle

In [16]:
```
from src.analysis.global_financial_cycle import (
    extract_global_factor, build_synthetic_gfc_panel, panel_gfc_regressio
)
from src.visualization.plots import plot_global_factor, plot_gfc_amplific

# Extract global factor
gfc = extract_global_factor(weekly, n_components=3)
print(f"PC1 explains {gfc['explained_variance_ratio'][0]:.1%} of variance
print(f"Top 3 PCs: {gfc['explained_variance_ratio'].sum():.1%}")

fig = plot_global_factor(gfc['global_factor'], save_name=None)
plt.show()
```

```
PC1 explains 52.2% of variance
Top 3 PCs: 54.8%
```

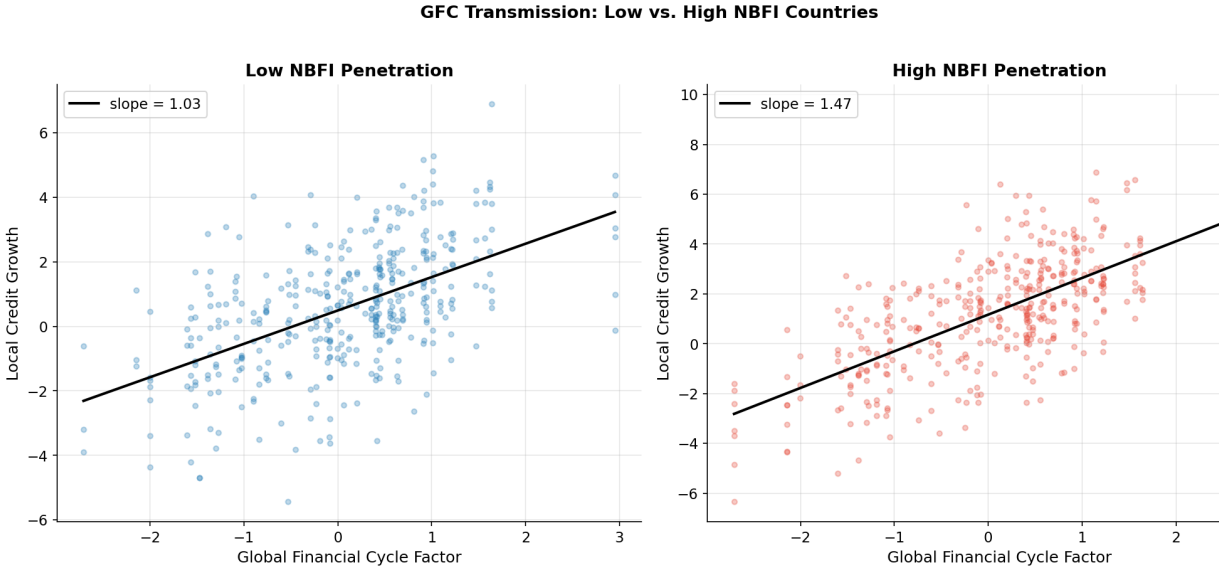


In [17]:
```
# GFC amplification regression
panel = build_synthetic_gfc_panel(true_amplification=0.5, n_countries=10,
reg = panel_gfc_regression(panel)

print("Panel Regression: y = credit_growth")
print("=" * 50)
for var in ['gfc_factor', 'nbfi_assets_pct_gdp', 'gfc_x_nbfi']:
    coef = reg.params.get(var, np.nan)
    pval = reg.pvalues.get(var, np.nan)
    stars = '***' if pval < 0.01 else '**' if pval < 0.05 else '*' if pva
```

```
    print(f"  {var:30s}  {coef:8.3f}  (p={pval:.3f}) {stars}")
print(f"  R-squared: {reg.rsquared:.3f}")
print(f"  N: {int(reg.nobs)}")
print("\nThe positive GFC x NBFI coefficient confirms that NBFI")
print("penetration amplifies the global financial cycle.")
```

```
Panel Regression: y = credit_growth
==================================================
  gfc_factor                          0.609  (p=0.000) ***
  nbfi_assets_pct_gdp                 0.007  (p=0.230)
  gfc_x_nbfi                          0.007  (p=0.000) ***
  R-squared: 0.482
  N: 800

The positive GFC x NBFI coefficient confirms that NBFI
penetration amplifies the global financial cycle.
```

In [18]:
```
fig = plot_gfc_amplification(panel, save_name=None)
plt.show()
```



Summary of Key Findings

1. **Network structure**: Bank-NBFI exposures are concentrated in a few G-SIBs
2. **Systemic risk**: Hedge funds and broker-dealers contribute most to ΔCoVaR
3. **DCC correlations**: Bank-NBFI correlations spike during stress
4. **Procyclicality**: Hedge funds show the strongest VaR-driven procyclical leverage
5. **Fire-sale amplification**: Adding NBFIs to a bank-only system significantly amplifies price de
6. **Subsector models**: LDI margin spirals, MMF runs, and HF deleveraging each create distin contagion channels to banks
7. **Global financial cycle**: Higher NBFI penetration amplifies the transmission of the global fac local credit conditions